Certified Algorithms for Combinatorial Optimization Problems

Martin Sokolov

March 31, 2025

Contents

1 Introduction **2**

2 Preliminaries **4**
2.1 Graph Theory 4
2.2 Combinatorial Optimization 4
2.2.1 NP-Complete Problems: Definitions and Variants 4
2.2.2 Approximation Algorithms and Complexity Classes 6
2.2.3 Baker's approach 6
2.2.4 Treewidth 8
2.2.5 Linear Programming and Randomized Rounding 9
2.3 Beyond the Worst-Case Analysis of Algorithms 9

3 Research Question **12**

4 (E)PTAS for the FVS-BCL Problem Using Baker's Technique **13**
4.1 Baker's technique for unweighted FVS-4CL on planar graphs 13
4.2 Baker's technique for the weighted FVS-4CL on planar graphs 18
4.3 Baker's technique for weighted FVS-BCL of arbitrary cycle length 21
4.4 Exact algorithm for FVS-4CL using MSOL 23
4.4.1 MSOL - a language for graph problems 23
4.4.2 Exact algorithm for unweighted FVS-4CL using MSOL 24
4.4.3 Exact algorithm for weighted FVS-4CL using MSOL 24
4.4.4 Exact algorithm for weighted FVS-BCL using MSOL 24
4.5 Exact algorithm for FVS-4CL using dynamic programming 25

5 m-stitching and FVS-BCL-m-stitching **26**
5.1 Algorithms for m-stitching and FVS-m-stitching 26

6 Certified Algorithm **33**
6.1 Certified Algorithm for FVS-4CL 33

1 Introduction

The Feedback Vertex Set (FVS) problem is central to the field of computer science. Karp's paper [1] showed that there is a reduction from Satisfiability to twenty-one problems, one of which is FVS. Because of the hardness of the problem, extensive research has been conducted in order to find a good approximation solution. The problem is **APX-complete** and the best approximation ratio so far is two and was first found by Becker and Geiger [2]. Around the same time, Bafna et al. [3] devised the first local ratio technique in order to arrive at the same approximation ratio. These algorithms were translated in primal-dual terms by Chudak et al. [4] who constructed a third algorithm which yield a 2-approximation both for the unweighted and weighted case of the FVS problem.

For planar graphs and H-Minor-Free graphs two approaches have yielded PTASs for many problems. Baker's technique [5] focused on approximation algorithms for **NP-Complete** problems on planar graphs, while Demaine and Hajiaghayi [6] generalized the shifting technique for the latter class of graphs. One of the known limitations of the shifting technique is that it does not handle non-local problems, such as the ones we are examining (FVS, Cycle Packing). In our paper, we impose restrictions on the problems in order to introduce a local property that allows us to devise a PTAS for the FVS-BCL problem. Baker's technique allows for the translation into the weighted version of the problem. We adapt the algorithm to work for the MWFVS-BCL all the while trying to follow the framework for devising certified algorithms established in [7, 8, 9].

The FVS problem has applications in the field of Bayesian networks. As we said, researchers turned to approximation algorithms and made use of the linear programming (LP) framework in order to get desirable ratios. In this paper, we make use of the existing literature on LP for FVS and analyse how to translate the results for the class of graphs with bounded cycle length. In Section 2.3, we familiarize the reader with two ways of designing certified algorithms. One way is by using convex relations. This motivates us to define the problem as an ILP, solve the LP relaxation and exploit a randomized rounding scheme to get the values for the approximate and co-approximate conditions, as defined in the same section. Those values are then used to calculate the γ for the $\gamma-$certified algorithm.

Finally, we consider the notions of Bilu-Linial stability [10] and certified algorithms [7]. In the last two paragraphs, we explained how we plan to translate existing techniques to get certified solutions of instances. In the final work, the last sections will be dedicated to proving that these algorithms are indeed certified.

Related Work

This work is inspired by Baker's seminal paper on designing PTASs in planar graphs [5]. There have been other techniques that generalize this approach, namely Demaine and Hajiaghayi [6]'s bidimensionality theory. The latter framework explores problems in H-Minor-Free graphs.

The notion of Bilu-Linial stability has given rise to much exploration of structured instances in many problems. In their paper, Bilu and Linial [10] offer the first evidence that stable instances are much easier than worst-case instances of problems. They propose an exact algorithm for $O(n)$-stable instances of Max Cut. This result was improved by Bilu et al. [11] who designed a $O(\sqrt{n})$ for the same problem. Further research yielded a general approach to analysing stable instances in Makarychev et al. [12]. They applied that approach to graph partitioning problems and designed $O(\sqrt{\log n}\log\log n)$-stable instances of Max Cut and 4-stable instances of Minimum Multiway Cut. The study of perturbation resilience of clustering problems is explored in [13, 14, 15]. On the Travelling Salesman Problem, Mihalák et al. [16] gave an algorithm for 1.8-stable instances. We refer the reader to the survey [17].

On the subject of Beyond the Worst-Case Analysis, we refer to Makarychev and Makarychev [7]. In that paper certified algorithms are first described in an attempt to bridge the gap between the worst-case and structured instances. Since then, researchers have made use of the definitions and results in that paper and applied them to other problems. Angelidakis et al. [8] explore the notion of stability and initiate the study

of certified algorithms for the Maximum Independent Set problem (MIS). They provide robust and certified algorithms for $(1+\epsilon)$-stable instances of planar MIS, for any fixed $\epsilon > 0$. Covering and Dominating in planar graphs was further explored by Bumpus et al. [9]. They provide $(1+\epsilon)$-certified algorithms for Dominating Set and H-Subgraph-Free-Deletion, which for any ϵ run in time $f(1/\epsilon) \cdot n^{O(1)}$ on minor-closed classes of graphs of bounded local treewidth.

2 Preliminaries

In this section, we provide the reader with the necessary information in order to follow the paper. We start with a reminder of Graph Theory. After that, we move to more advanced definitions in the field of Combinatorial Optimization. Finally, we get on topic by introducing perturbation resilience and certified algorithms from the field of Beyond the Worst-Case Analysis of Algorithms.

2.1 Graph Theory

In this section, the authors refer to Bondy and Murty [18]. A *graph* G is an ordered pair denoted as $(V(G), E(G))$. The set $V(G)$ is called a set of *vertices* and the set $E(G)$ is a set of *edges*. The incidence function ψ_G associates each edge with an unordered pair of (not necessarily unique) vertices. The ordered pair and the incidence function define the graph G. We are going to use n as the cardinality of the set V and m for the set of edges E. With each edge e of G, let there be associated a real number $w(e)$, called its *weight*. Then G, together with the weighting function $w : E \to \mathbb{R}$, is called a *weighted graph*, and denoted (G, w).

A graph can be *embedded in the plane* or is *planar* if it can be drawn on the plane in a way that edges intersect each other only at their ends. Such a drawing is called a *planar embedding* of the graph. A graph G is called *outerplanar* if it has a planar embedding $\widetilde{G}$ in which all the vertices lie on the boundary of the outer face. This is also called a *1-outerplanar graph*. Two forbidden minors of that class are the K_4 and $K_{2,3}$ graphs. In the next subsection, we familiarize the reader with Baker's approach [5]. Here, we define a *ℓ-outerplanar* to be a graph that has a planar embedding in which the vertices belong to at most ℓ concentric layers. Every planar graph is *ℓ-outerplanar* for some ℓ. Given a graph G, we look for the *ℓ-outerplanar* embedding of G for which ℓ is minimal.

2.2 Combinatorial Optimization

In this section, we refer to the book on Combinatorial Optimization by Korte and Vygen [19] and Williamson and Shmoys [20]. Some definitions can be traced to the end of the book by Cygan et al. [21].

2.2.1 NP-Complete Problems: Definitions and Variants

We are going to be looking at a couple of problems, namely the Feedback Vertex Set, the Cycle Packing problem and the Travelling Salesman Problem. All of those problems belong to the class of **NP-complete** problems. The decision variant of the FVS problem can be defined as:

Definition 2.1: Feedback Vertex Set (FVS)

Input: A graph G and an integer ℓ.
Question: Does there exist a set X of at most ℓ vertices of G such that $G - X$ is a forest?

The weighted optimization version of the problem can be defined as:

Definition 2.2: Minimum Weight Feedback Vertex Set (MWFVS)

Instance:
- An undirected weighted graph (G, w), where $G = (V, E)$ and $w : V \to \mathbb{R}$.

Task: Find a vertex set $X \subseteq V$ such that:
- $G \setminus X$ is acyclic (i.e., a forest)
- The sum of the weights of the vertices in X is minimized.

In some sense, the Cycle Packing problem is dual to FVS. We use the definition from the blue book of Cygan et al. [21].

Definition 2.3: Cycle Packing

Input: A graph G and an integer ℓ.
Question: Does there exist in G a family of ℓ pairwise vertex-disjoint cycles?

The other problem we inspect is an edge-optimization problem, namely the Travelling Salesman Problem (TSP).

Definition 2.4: Travelling Salesman Problem (TSP)

Instance:
- A set of cities $C = \{c_1, c_2, \ldots, c_n\}$.
- A distance function $w : C \cdot C \to \mathbb{R}$ that gives the distance between each pair of cities.

Task: Find a permutation π of the cities such that:
- The total distance travelled is minimized, i.e., minimize

$$\sum_{i=1}^{n-1} w(c_{\pi(i)}, c_{\pi(i+1)}) + w(c_{\pi(n)}, c_{\pi(1)}).$$

We define the problems FVS-BCL, MWFVS-BCL and Cycle Packing-BCL to be the problems that cover and pack cycles of bounded length. The 'BCL' stands for bounded cycle length. We give the following definitions:

Definition 2.5: FVS-BCL

Input: A graph G, integer ℓ and an integer r.
Question: Does there exist a set X of at most ℓ vertices such that every cycle of length at most r contains a vertex in X?

The following is the weighted version of the problem; we define it as an optimization task.

Definition 2.6: MWFVS-BCL

Instance:
- An undirected weighted graph (G, w), where $G = (V, E)$ and $w : V \to \mathbb{R}$ and an integer r.

Task: Find a vertex set $X \subseteq V$ such that:
- Every cycle in $G \setminus X$ of length $\leq r$ has a vertex in X.
- The sum of the weights of the vertices in X is minimized.

Finally, we define the maximization problem with an additional constraint:

Definition 2.7: Cycle Packing-BCL

Input: A graph G, an integer ℓ and an integer r.
Question: Does there exist in G a family of ℓ pairwise vertex-disjoint cycles with each of them having length $\leq r$?

2.2.2 Approximation Algorithms and Complexity Classes

As we mentioned, all the problems are **NP-complete**, so we turn to *approximation algorithms* in order to cope with the hardness of the problems. Let Π be an optimization problem with nonnegative weights and $k \geq 1$. A *k-factor approximation algorithm* for Π is a polynomial-time algorithm A for Π, such that

$$\frac{1}{k} \cdot OPT(I) \leq A(I) \leq k \cdot OPT(I)$$

for all instances I of Π. The first inequality shown is true for maximization problems, while the second one holds for minimization problems. We say A has a performance ratio of k. If a problem falls into that category, we say that it is **APX-complete**.

In the last paragraph, we mentioned the class of problems **APX** regarding approximation algorithms. Now we turn to another class of approximation algorithms. **PTAS** is a class of algorithms that work in polynomial time and get close to an optimal solution with a trade-off.

Definition 2.8: Polynomial-Time Approximation Scheme (PTAS)

A polynomial-time approximation scheme (**PTAS**) is a family of algorithms A_ϵ where there is an algorithm for each $\epsilon > 0$, such that A_ϵ is a $(1+\epsilon)$-approximation algorithm (for minimization problems). The running time of the algorithm A_ϵ is polynomial, but can depend arbitrarily on $1/\epsilon$. This dependence could be exponential or even worse. Thus an algorithm running in time $O(n^{1/\epsilon})$ counts as a PTAS.

Every problem with a PTAS is also in APX. The relation between these two classes is similar to the one between **P** and **NP**. For more on the topic, such as APX-completeness, L-reductions and the PCP-theorem, we refer to Korte and Vygen [19].

An important distinction in the complexity zoo is made in Cesati and Trevisan [22]. To the best of the knowledge of the author, this is the paper that describes what an efficient PTAS is.

Definition 2.9: Efficient Polynomial-Time Approximation Scheme (EPTAS)

An efficient PTAS yields a $(1+\epsilon)$ approximation scheme and has a running time of $f(\epsilon) \cdot n^c$ where f is an arbitrary function and c is a constant.

2.2.3 Baker's approach

In this subsection, we are going to describe the Shifting technique. It is described in Baker's seminal paper [5]. What she describes is a general algorithm that partitions the graph into a disjoint union of levels of vertices. By analysing slices of these levels and, more specifically, the overlap between these slices, one obtains an approximation ratio for the solution to the optimization problem. The slices make up several levels that are specifically chosen for a problem and it should hold that an exact solution can be found on one slice. It must also hold that this solution can be found in optimal time.

There is also another way to view this approach. Instead of analysing the overlap between slices, one could partition the graph by removing said overlap. This way we get a disjoint union which does not sum up to

the total solution, but we can argue that we did not stray away so much from the optimum solution. This approach works for hereditary maximization problems like Independent Set, Max-Cut, Maximum P-Matching, etc.

We are going to give two examples. The first is the general shifting technique and the example problem for it will be Vertex Cover. Then we will explore the second perspective and take the Independent Set problem as an example. It is important to note that the general technique is much more powerful than partitioning by removal of vertices. With Vertex Cover, we can still use the second approach and get a PTAS, but the same does not hold for the Dominating Set problem. However, using the general shifting technique, we can obtain a PTAS for Dominating Set in planar graphs. This was done by Marzban and Gu [23]. They designed a PTAS with an approximation ratio of $(1+2/\ell)$ in $O(2^\ell \cdot n)$ time. The two in the approximation ratio appears because of the overlap needed for Dominating Set between slices, which is two levels of overlap.

What follows is a brief explanation of Baker's shifting technique with Vertex Cover as an example.

Algorithm 1 Baker's shifting technique

Require: Planar graph $G = (V, E)$, parameter ℓ
Ensure: $(1+\epsilon)$-approximation for Vertex Cover
1: Perform BFS from some arbitrary vertex r
2: $S \leftarrow \emptyset$
   i: *shift*; j: *slice*
3: **for** each $i = 0$ to $\ell - 1$ **do**
4: Let $G_{i,j}$ be the subgraph induced on vertices at levels $j \cdot \ell + i$ through $(j+1) \cdot \ell + i$ for $0 \leq i < \ell$.
5: $\forall j$ compute $S_{i,j}$, the min. VC of $G_{i,j}$.
6: $\forall j$ let $S_i = \bigcup_j S_{i,j}$
7: $S \leftarrow S \cup S_i$
8: **end for**
9: **return** S_{i*} from S with minimum cardinality

Analysis of the algorithm:

Let V_i be the set of vertices at levels $i \mod \ell$:

$$V_i = \{v \in V \mid \text{level}(v) \equiv i \mod \ell\}$$

Let OPT_i denote $OPT \cap V_i$. We notice that OPT_i are disjoint and $OPT = \bigcup_i OPT_i$. Therefore, we can make the following argument:

1. $\exists q : |OPT_q| \leq \frac{1}{\ell} \cdot OPT = \epsilon |OPT|$
2. $S_q \leq \sum_j S_{q,j}$, for some shift q we have this inequality because in step 5. of the algorithm we define the set to be a union over all slices and some vertices may appear twice in the boundary layers.
3. $\sum_j S_{qj} \leq \sum_j |OPT \cap V(G_{q,j})|$, we know that $S_{q,j}$ is a min. VC for $G_{q,j}$, while the optimal solution may choose different vertices, especially on the boundary for the global optimality of the solution.
4. $\sum_j |OPT \cap V(G_{q,j})| \leq |OPT| + |OPT_q|$, if we consider the graphs $\{G_{q,j}\}$, then we can argue that each vertex appears once, except those at levels $q \mod \ell$, which appear twice.

From the first, second and fourth step we can conclude that:

$$S_q \leq |OPT| + |OPT_q| \leq (1+\epsilon) \cdot |OPT|$$

Now we take a look at the second approach we described.

Algorithm 2 Baker's technique with removal

Require: Planar graph $G = (V, E)$, parameter ℓ
Ensure: $(1 - \epsilon)$-approximation for MIS
1: $S \leftarrow \emptyset$
2: Perform BFS from some arbitrary vertex r
3: $\ell \leftarrow 1/\epsilon$
4: Label each vertex according to its layer $i \mod \ell$: $V_1, V_2, \ldots V_{\ell-1}$
5: **for** each $i = 0$ to $\ell - 1$ **do**
6: Let $G_i \leftarrow G \backslash V_i$
7: Let $G_1, G_2, \ldots G_j$ be the connected components of G_i
8: $\forall j$ compute the MIS $S_{i,j}$ of $G_{i,j}$
9: $S_i \leftarrow \bigcup_j S_{i,j}$
10: $S \leftarrow S \cup S_i$
11: **end for**
12: **return** S_{i*} from S with minimum cardinality

Consequence of the algorithm:

1. for $\ell = 1/\epsilon$ compute partition $P_1, \ldots, P_k$.
2. for some q $(1 \leq q \leq \ell)$, $|OPT \cap P_q| \leq \frac{1}{\ell} \cdot |OPT| = \epsilon \cdot |OPT|$ because of the pigeonhole principle
3. hence, $|OPT \cap (G - P_q)| \geq (1 - 1/\ell) \cdot |OPT| = (1 - \epsilon) \cdot |OPT|$
4. return $max_i |\text{MIS}(G - P_i)|$

We skip much of the analysis that we did for the shifting technique, however, we argue that for this particular problem for some P_q, a small fraction of the solution resides within that partition. Therefore, we can forget about that part and the rest of the solution will still be an independent set of bounded size.

The first approach is more general and powerful for constructing algorithms, but it doesn't apply well to the Dominating Set problem. The key issue is that the removal technique relies on intersecting an optimal solution with a subgraph, expecting it to remain valid for the subgraph. This fails for Dominating Set, since a vertex in the subgraph could still be dominated by a vertex outside the subgraph.

It is shown that the general shifting technique can be used to obtain a PTAS for Dominating Set. The interesting part is that in order to properly analyse the solution, one must make the slices overlap on two levels. We refer to Marzban and Gu [23] for the explanation as to why the last is true and how to obtain the $(1 + 2/\ell)$ approximation ratio.

2.2.4 Treewidth

According to Diestel [24, pp. 354-355] the notions of tree decomposition and treewidth were introduced by Halin [25] under the name of S-functions and Robertson and Seymour [26]. We give brief definitions of these concepts and then discuss bounded local treewidth and linear local treewidth. The last two notions are important for the main theorem we work with, namely Bumpus et al. [9, Theorem 1.7].

From Diestel [24, Section 12.3] we have the following definition of a tree decomposition.

Definition 2.10: Tree Decomoposition

Let G be a graph, T a tree, and let $\mathcal{V} = (V_t)_{t \in T}$ be a family of vertex sets $V_t \subseteq V(G)$ indexed by the vertices t of T. The pair $(T, \mathcal{V})$ is called a *tree decomposition* of G if it satisfies the following conditions:
(T1) $V(G) = \bigcup_{t \in T} V_t$.
(T2) For every edge $e \in E(G)$, there exists a $t \in T$ such that both ends of e lie in V_t.
(T3) If $t_1, t_2, t_3 \in T$ satisfy t_2 lies on the unique path between t_1 and t_3 in T, then

$$V_{t_1} \cap V_{t_3} \subseteq V_{t_2}.$$

The *width* of a tree decomposition (T, V) is the size of its largest set V_t minus one. The *treewidth* $\mathrm{tw}(G)$ of a graph G is the minimum width among all possible tree decompositions of G.

Definition 2.11: Bounded local treewidth

A graph G has bounded local treewidth if the subgraph induced by vertices at distance at most d is a function of d ($f : \mathbb{N} \to \mathbb{R}$) and not n.

Furthermore, if there exists $\lambda \in \mathbb{R}$, such that f can be defined as $f : r \to \lambda \cdot r$, then G would have linear local treewidth.

Building on the work of Eppstein [27], Demaine and Hajiaghayi [28] prove that apex-minor-free graphs have linear local treewidth. Apex graphs are graphs that can be made planar by the removal of a single vertex. From this follows that planar graphs also have linear local treewidth.

2.2.5 Linear Programming and Randomized Rounding

One way of approximating solutions is done through Linear Programming (LP). For a more detailed review of LP, the reader can consult Korte and Vygen [19, Chapter 3]. One solves a linear relaxation, which provides fractional values as solutions. Afterwards, the fractional solution is rounded to an integral one, but rather than doing it deterministically, this process is done in a randomized fashion. This technique is called *randomized rounding*. This framework provides a way to obtain certified algorithms, for a more in depth look for randomized rounding the reader should consult Makarychev and Makarychev [7].

2.3 Beyond the Worst-Case Analysis of Algorithms

In this section we explore perturbation-resilience, also known as γ-*stability* or Bilu-Linial stability. The authors refer to the original paper of the two authors Bilu and Linial [10] as well as a survey on the matter by Makarychev and Makarychev [17]. In a later work, Makarychev and Makarychev define γ-*certified algorithms* [7]. In the final paragraph, we take a look back at combinatorial optimization problems through the lens of constraint satisfaction.

The following definitions for γ-perturbation are for edge-optimization and vertex-optimization problems, respectively. Afterwards, we are going to define stability and certified algorithms for vertex-optimization problems only.

Definition 2.12: γ-perturbation for Edge-Optimization problems

Consider a weighted graph $G = (V, E, w)$ with a set of edge weights w_e. An instance $G' = (V, E, w')$ is a γ-perturbation ($\gamma \in \mathbb{R}_{\geq 1}$) of G if $w(e) \leq w'(e) \leq \gamma \cdot w(e)$; that is, if we can obtain a perturbed instance from the original by multiplying the weight of each edge by a number between 1 and γ (the number may vary for each edge).

From now on, the definitions will concentrate on Vertex-Optimization problems.

Definition 2.13: γ-perturbation for Vertex-Optimization problems

Consider a weighted graph (G, w). For any $\gamma \in \mathbb{R}_{\geq 1}$ a γ-perturbation of the weight function $w : V \to \mathbb{R}$ is a function $w' : V \to R$ satisfying $w(v) \leq w'(v) \leq \gamma \cdot w(v)\ \forall v \in V$.

Definition 2.14: γ-stability

For any $\gamma \in \mathbb{R}_{\geq 1}$, a γ-stable instance (G, w) of a *vertex-minimization* problem Π is an instance admitting a unique optimal solution S which remains optimal even under γ-perturbations of (G, w).

Definition 2.15: Certified Algorithm

A γ-certified solution to an instance (G, w) of a weighted vertex-optimization problem Π is a pair (S, w') where w' is a γ-perturbation of w and S is an optimal solution on (G, w'). A γ-certified algorithm for Π maps instances of Π to γ-certified solutions.

Definition 2.16: Combinatorial Optimization problems (Constraint Satisfaction)

An instance of a *combinatorial optimization* problem is specified by a set of feasible solutions $\mathcal{S}$ (the solution space) a set of constraints $\mathcal{C}$ and constraint weights $w_c > 0$ for $c \in \mathcal{C}$. Typically the solution space is exponential in size (as is the case with MWFVS with the number of cycles in a graph) and is not provided explicitly. We say that a feasible solution $s \in \mathcal{S}$ satisfies a constraint $c \in \mathcal{C}$ if $c(s) = 1$.
Consider the minimization and maximization objectives.

- The minimization objective is to minimize the total weight of the unsatisfied constraints: find $s \in \mathcal{S}$ that minimizes $\sum_{c \in C} w_c \cdot (1 - c(s)) = w(\mathcal{C}) - val_{\mathcal{I}}(s)$, where $w(\mathcal{C}) = \sum_{c \in \mathcal{C}} w(c)$ is the total weight of the constraints.
- The maximization objective is to maximize the total weight of the satisfied constraints: find $s \in \mathcal{S}$ that maximizes $val_{\mathcal{I}}(s) = \sum_{c \in \mathcal{C}} w(c) \cdot c(s)$.

Now we familiarize the reader with the two ways of designing certified algorithms as presented in [7]. The first is a general framework that (1) starts with an arbitrary solution and (2) iteratively improves it. The improvements in step 2, however, are not necessarily local, though it may look like a local search technique. The following is taken from the paper on certified algorithms where it is presented as Task 5.13.

Task 2.17: General framework for γ-certified algorithms

Assume we are given an instance $\mathcal{I}(\mathcal{S}, \mathcal{C}, w)$, a partition of its constraints $\mathcal{C} = \mathcal{C}_1 \cup \mathcal{C}_2$ and a parameter γ. The task is to either:

- Option 1: find a better $s \in \mathcal{S}$, such that $\gamma \cdot \sum_{c \in \mathcal{C}_1} w_c c(s) > \sum_{c \in \mathcal{C}_2} w_c (1 - c(s))$, or
- Option 2: to report that for every $s \in \mathcal{S}$: $\sum_{c \in \mathcal{C}_1} w_c c(s) \leq \sum_{c \in \mathcal{C}_2} w_c (1 - c(s))$.

In this procedure, $\mathcal{C}_1$ and $\mathcal{C}_2$ are the sets of constraints that are currently unsatisfied and satisfied, respectively. If $\gamma = 1$, then Option 1 would try to find a solution where the weight of the unsatisfied constraints is strictly greater than the weight of the unsatisfied by s. Therefore, Option 1 finds a better solution than the current one, while Option 2 reports that there is no solution better than s. The options are not mutually exclusive.

For the second approach, we refer the reader to the paper of Makarychev and Makarychev [7] as the explanation is quite long and needs to be understood with the examples given.

3 Research Question

In this work, we explore several questions. The first two are of most significance, while the problems that follow will be tackled in what time is left.

- Explore the FVS-BCL problem as defined in Definition 2.5 for planar graphs.
  - To that end we are going to make use of Baker's shifting technique [5] as presented in Algorithm 1.
  - By using the modified version with an overlap between layers of $\lfloor \frac{r}{2} \rfloor + 1$, we capture the essence of breaking cycles of bounded length r.
  - Baker's technique is known to work for problems that involve local conditions on nodes and edges and our imposed restrictions on the FVS allow us to deploy this framework.
  - We extend this method to devise a PTAS for the weighted version of the problem, MWFVS-BCL, as defined in Definition 2.6, for planar graphs.
- Initiate the study of certified algorithms for MWFVS-BCL. We provide robust and certified algorithms for $(1+\epsilon)$-stable instances of planar MWFVS-BCL, for any fixed $\epsilon > 0$.
- Future Directions:
  - Investigating the Cycle Packing-BCL problem. As a hereditary maximization task, we make use of the alternative approach to Baker's technique, namely Algorithm 2 with removal.
  - Exploring the translation of PTASs for the TSP to $(1+\epsilon)$-certified algorithms.

4 (E)PTAS for the FVS-BCL Problem Using Baker's Technique

In this section we will demonstrate how to obtain an EPTAS for the unweighted and weighted versions of FVS-BCL. We begin by looking at the unweighted version and argue about the correctness and efficiency of our algorithm. Then we prove that using treewidth and Monadic Second Order Logic (MSOL) one can exactly solve the 4-cycle-breaking problem in one subgraph. Finally, we show that the running time is indeed polynomial in the size of the input.

Before we start, we introduce some notation that we are going to use from now on. When we are looking to break 4-cycles, we will note that the problem as FVS-4CL. This will be the notation used in subsections 4.1 and 4.2, i. e. regardless of whether or not the problem is weighted. In subsection 4.3 we go back to calling the problem Π - FVS-BCL as we have arbitrary cycle length ρ. In further chapters we also make use of that notation and if mentioned explicitly, one should regard FVS-4CL as the weighted version of the problem.

4.1 Baker's technique for unweighted FVS-4CL on planar graphs

In this subsection we provide the necessary definitions, lemmas and theorems to demonstrate how to obtain an EPTAS for the problem we defined in 2.5, namely FVS-4CL. In the preliminaries we gave two examples of how to construct algorithms using Baker's technique and how to further analyse them. In the same fashion we now construct Algorithm 3 and reason for its effectiveness.

In the following algorithm we use the letter ℓ for the outerplanarity of each subgraph, i.e. each $G_{i,j}$ that we define on line 4 in Algorithm 3 is ℓ-outerplanar.

Algorithm 3 Baker's technique for the unweighted FVS-4CL

Require: Planar graph $G = (V, E)$, parameter ℓ
Ensure: $(1+\epsilon)$-approximation for FVS-4CL
1: Perform BFS from some arbitrary vertex r
2: $S \leftarrow \emptyset$
   i: *shift*; j: *slice*
3: **for** each $i = 0$ to $\ell - 1$ **do**
4: Let $G_{i,j}$ be the subgraph induced on vertices at levels $j \cdot \ell + i$ through $(j+1) \cdot \ell + i + 1$ for $0 \leq i < \ell$.
5: $\forall j$ compute $S_{i,j}$, the min. unweighted FVS-4CL of $G_{i,j}$ using Algorithm 4 as a subroutine.
6: $\forall j$ let $S_i = \bigcup_j S_{i,j}$
7: $S \leftarrow S \cup S_i$
8: **end for**
9: **return** S_{i*} from S with minimum cardinality

In the analysis of the algorithm we are going to need three lemmas in order to prove that Algorithm 3 obtains a $(1 + 2/\ell)$-approximation of the optimum solution.

Lemma 4.1: Optimum solution bound (Unweighted Case)

Let $S_{i,j}$ be an optimum solution for $G_{i,j}$ for some shift i and slice j, let $F \equiv OPT$ be the optimum solution in G and let $F_{i,j} = G_{i,j} \cap F$. In the unweighted case we claim that:

$$|S_{i,j}| \leq |F_{i,j}|.$$

We will say that $F_{i,j}$ is a FVS-4CL of $G_{i,j}$, not necessarily the one of minimum cardinality.

For now we do not argue about how we obtain an optimum solution. In subsection 4.4 we show how to get an exact solution, but for now we argue that the inequality in Lemma 4.1 holds in each subgraph $G_{i,j}$.

Proof. We first argue that in each subgraph $G_{i,j}$, the set $F_{i,j}$ contains a vertex that breaks every 4-cycle.

1. Let us fix a slice and shift a and b and take a look at the subgraph $G_{a,b}$.
2. For $G_{a,b}$, using Algorithm 3 we can get $S_{a,b}$ and we have $F_{a,b}$.
3. Notice that, much like in the Vertex Cover analysis, no vertex in $V(G)\backslash V(G_{a,b})$ can break any 4-cycle that is contained within $G_{a,b}$. Therefore $F_{a,b}$ must contain a vertex that hits every 4-cycle or it will not be a solution in $G_{a,b}$ and subsequently in G.
4. From this and the fact that $S_{a,b}$ is the optimum solution for $G_{a,b}$ we conclude that:

$$|S_{a,b}| \leq |F_{a,b}|.$$

for some (a, b), therefore it holds $\forall(i, j)$.

□

One must confront the fact that notation is abused throughout the paper, especially when it comes down to j, the slice variable. When we say that a statement holds for all values i and j, we mean that it holds for the subgraphs that these variables outline. We will sketch an example to deepen our understanding of Lemma 4.1. Moreover, we are going to explore one avenue that gives rise to an inequality in the lemma.

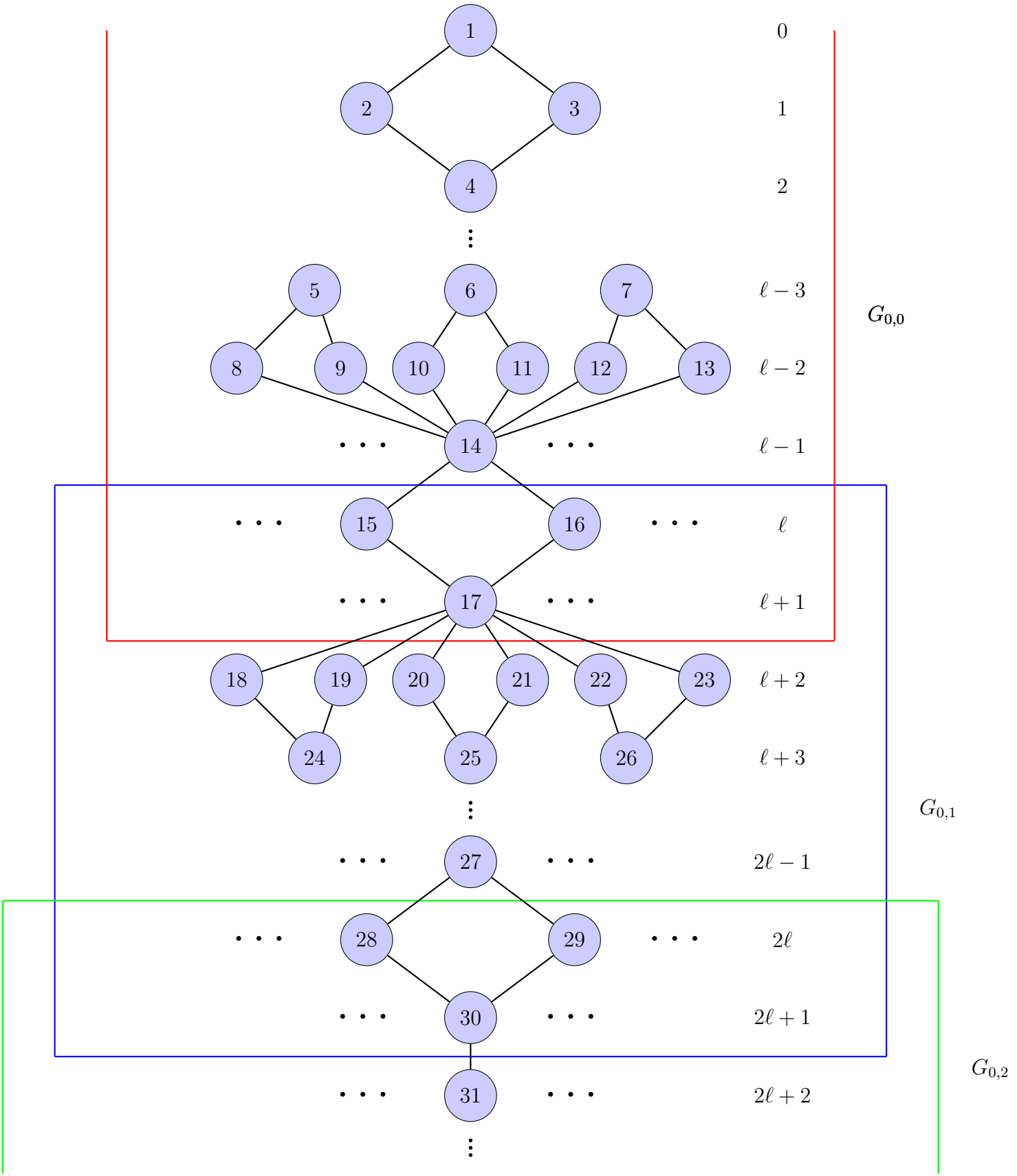

Figure 1: BFS tree, showing the distance from the root for each layer.

Let G look like the graph we sketched on the previous page. The vertical dots help us clarify the levels

while the horizontal signify that there may be other vertices of interest. However, there are vertices we would like to inspect, specifically vertices with labels 14 and 17. Let $v_{14}, v_{17} \in F$. What follows is that $v_{14}, v_{17} \in F_{0,0}$. The cycles that give us reason to claim that $v_{14} \in F$ are all within $G_{0,0}$. Therefore, it holds that Algorithm 3 will have $v_{14} \in S_{0,0}$. However, the same does not hold for v_{17}, the cycles that it is a part of are in $G_{0,1}$ and thus $v_{17} \notin S_{0,0}$, while $v_{17} \in F_{0,0}$. This example should not distract the reader from the fact the real reason that $|S_{0,0}| \leq |F_{0,0}|$ is because $S_{0,0}$ is by definition the minimum 4-cycle-breaking set for $G_{0,0}$, while $F_{0,0}$ is, as we previously argued, a feasible 4-cycle-breaking set.

What follows from Lemma 4.1 is a result that holds for any shift i.

Result 4.2: Optimum solution bound for the whole graph (Unweighted Case)

Let S_i be the union of optimal solutions defined on line 6 of Algorithm 3 for some shift i. Let $F \equiv OPT$ be the optimum solution in G and let $F_{i,j} = G_{i,j} \cap F$. For those sets it holds that:

$$|S_i| \leq \sum_j |S_{i,j}| \leq \sum_j |F_{i,j}|.$$

The first inequality in Result 4.2 stems from the fact that some vertices in the $S_{i,j}$ sum may appear twice. This pertains to vertices that are located on the boundary. For example if the $v_{30} \in S_{0,1} \wedge v_{30} \in S_{0,2}$, then it would be counted once in S_0, but twice in $\sum_j S_{0,j}$. The second inequality is a result from Lemma 4.1 where we argue that $S_{i,j}$ is an optimum solution for $G_{i,j}$ while $F_{i,j}$ is a 4-cycle-breaking set, but not one of necessarily minimum cardinality.

In Lemma 4.4 we look at the bound of the union of the $F_{i,j}$ sets. Before that, however, we make an argument for the vertices on the boundaries in Lemma 4.3.

Before we continue with the next lemma, we define what in [9, Definition 4.5] is described as modular slices. In the preliminaries section the algorithms () had one layer of overlap. Now we need a way to map to all s-consequent layers where s is a width parameter. For Fig. 2, imagine we ran a BFS algorithm from an arbitrary vertex and the graph was decomposed into nine layers.

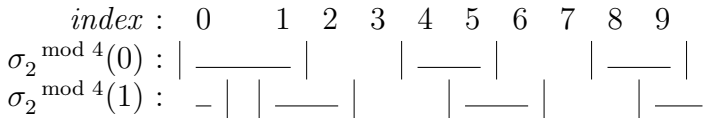

Figure 2: Modular slices

The notation $\sigma_s{}^{\mod \ell}(i)$ means that we take all of the vertices that are on levels that start at level $i \mod \ell$ of width s. In our case (for the FVS-4CL problem), we wanted two layers of overlap. Therefore, s is equal to two.

If we are examining the set $\sigma_2^\ell(i)$, then we are in fact talking about the sets F_i and F_{i+1}, notice that $F_{i+1 \mod \ell}$ may wrap around and be F_0.

Lemma 4.3: Bound for the vertices on the boundaries (Unweighted Case)

Let $F_i = F \cap \{\text{vertices at levels } i \bmod \ell\}$. The sets F_i are disjoint and $\bigcup_i F_i = F$.
We claim that:

$$\exists q \in \{0, 1, \ldots, \ell - 1\} : |\sigma_2^\ell(q)| \leq \frac{2}{\ell} \cdot |F|$$

Proof. Assume, by way of contradiction, that:

$$\forall i \in \{0, 1, \ldots, \ell - 1\} : |\sigma_2^\ell(i)| > \frac{2}{\ell} \cdot |F| \tag{1}$$

As we said the sets F_i are disjoint, i. e. $\forall i, j (i \neq j)$, $F_i \cap F_j = \emptyset$. Also, $F = \cup_i F_i$. We make the following arguments:

1. If inequality 1 holds, then $\sum_{i=0}^{\ell-1} |\sigma_2^\ell(i)| > \sum_{i=0}^{\ell-1} \frac{2}{\ell}|F| = 2 \cdot |F|$.
2. Now, we argue for the LHS of the inequality in the previous step. In the sum $\sum_{i=0}^{\ell-1} |\sigma_2^\ell(i)|$, $\forall i \in \{0, 1, \ldots, \ell - 1\} : F_i$ is counted exactly twice. Therefore $\sum_{i=0}^{\ell-1} |\sigma_2^\ell(i)| = 2 \cdot |F|$
3. From the contradiction in step two, we argue that there indeed $\exists q : q \in \{0, 1, \ldots, \ell - 2\}$ by the *pigeonhole principle*, such that $|\sigma_2^\ell(q)| \leq \frac{2}{\ell} \cdot |F|$.

□

Now we argue for the size of the sets $F_{i,j}$. It is natural that we should be able to tie their size to the size of the optimal solution F. We will use Lemma 4.3 to do so. Before we state the lemma, the reader should keep in mind that we will abuse notation in order to avoid the loaded symbol ${\sigma_s}^{\bmod \ell}$. Whenever we refer to F_i and F_{i+1}, one should keep in mind that the second term may wrap around and become F_0. This definition is instructive and we will come back to it in the weighted case and more importantly in the arbitrary cycle length case in 4.10.

Lemma 4.4: Bound for the cardinality of the intersection sets (Unweighted Case)

Let $F \equiv OPT$ be the optimal solution and $F_{i,j} = F \cap G_{i,j}$ be the intersection sets with the subgraphs. Then we have:

$$\sum_j |F_{i,j}| \leq |F| + \frac{2}{\ell} \cdot |F|$$

for some specific integer i.

Proof. If we look at Fig. 1, then we notice that any vertex that is located in the overlap layers would be counted twice in the sum $\sum_j |F_{i,j}|$. The interior vertices are counted once towards the optimum solution sum. Therefore, we have that:

1. $\sum_j |F_{i,j}| = |F| + |\{\text{vertices on the boundary}\}| = |F| + |F_i| + |F_{i+1}|$ for some $i \in \{0, 1, \ldots, \ell - 1\}$.
2. For the RHS of the equation in step 1. we argue that:

$$\exists i^* : |F_{i^*}| + |F_{i^*+1}| \leq \frac{2}{\ell} \cdot |F|$$

   using Lemma 4.3.
3. For that i^* it holds that $\sum_j |F_{i^*,j}| \leq |F| + \frac{2}{\ell}|F|$.

□

Finally, we chain the inequalities presented in Lemma 4.1, Result 4.2 and Lemma 4.4, and get that for some slice i^*, it holds that:

$$|S_{i^*}| \leq \sum_j |S_{i^*,j}| \leq \sum_j |F_{i^*,j}| \leq |F| + \frac{2}{\ell} \cdot |F| = (1 + \frac{2}{\ell}) \cdot |F|$$

.

The inequality $S_{i^*} \leq (1+\frac{2}{\ell}) \cdot |F|$, gives us that the solution produced by Algorithm 3 has the approximation ratio $(1 + \frac{2}{\ell})$ for the planar unweighted FVS-4CL.

Running time:

Lines 1 and 9 of Algorithm 3 can be computed in linear time $O(n)$. The loop that starts on line 3 has a running time that is dominated by step 5 where we call Algorithm 4 as a subroutine. That algorithm takes as input subgraphs that are $\ell + 2$-outerplanar. Using a result from Bodlaender [29, Theorem 83], we get that the treewidth of each of those subgraphs is $3 \cdot \ell + 6 - 1 = 3 \cdot \ell + 5$. Finally, using Theorem 4.12, we conclude, that the for loop takes $O(f(3\ell+5) \cdot \ell \cdot n)$ for some computable function f. Therefore, Algorithm 3 has a running time of $O(n) + O(f(3\ell + 5) \cdot \ell \cdot n) + O(n) = O(f(3\ell + 5) \cdot n)$.

4.2 Baker's technique for the weighted FVS-4CL on planar graphs

In this subsection we look at the weighted version of the problem. Firstly, we construct Algorithm 4 which differs from Algorithm 3 only on step 5. On line 5 we construct Algorithm 5 and use it as a subroutine to get the solution we want.

Algorithm 4 Baker's technique for the weighted FVS-4CL

Require: Planar graph $G = (V, E)$, parameter ℓ, $w : V \to \mathbb{R}$
Ensure: $(1 + \epsilon)$-approximation for FVS-4CL
1: Perform BFS from some arbitrary vertex r
2: $S \leftarrow \emptyset$
i: *shift*; j: *slice*
3: **for** each $i = 0$ to $\ell - 1$ **do**
4: Let $G_{i,j}$ be the subgraph induced on vertices at levels $j \cdot \ell + i$ through $(j + 1) \cdot \ell + i + 1$ for $0 \leq i < \ell$.
5: $\forall j$ compute $S_{i,j}$, the min. weighted FVS-4CL of $G_{i,j}$ using Algorithm 5 as a subroutine.
6: $\forall j$ let $S_i = \bigcup_j S_{i,j}$
7: $S \leftarrow S \cup S_i$
8: **end for**
9: **return** S_{i*} from S, such that $w(S_{i^*}) \leq w(S_i), \forall i \in \{1, 2, \ldots, \ell - 1\}$.

Much like the proof we did in subsection 4.1, we start off with finding a bound for the optimum solution for each subgraph.

Lemma 4.5: Optimum solution bound (Weighted Case)

Let $G = (V, E)$ be a planar graph and $w : V \to \mathbb{R}$ be a weight function. Let $S_{i,j}$ be an optimum solution for $G_{i,j}$ for some shift i and slice j, let $F \equiv OPT$ be the optimum solution in G and let $F_{i,j} = G_{i,j} \cap F$. In the weighted case we claim that:

$$w(S_{i,j}) \leq w(F_{i,j}).$$

Proof. The proof of Lemma 4.5 is presented in three steps. Let us fix the integer arguments for shift and slice. We take a to be the offset (shift) and look at an arbitrary slice b. Note that when we look at a slice b, we mean that we look at the subgraph $G_{a,b}$ induced on vertices at levels $b \cdot \ell + a$ through $(b+1) \cdot \ell + a + 1$.

1. Firstly, one needs to be sure that no vertex v, such that $v \in V(G) \backslash V(G_{a,b})$ can break any 4-cycle that is contained within $G_{a,b}$.
2. From step 1 and the definition of $F_{a,b}$, it follows that $F_{a,b}$ is a feasible solution of the 4-cycle-breaking problem within $G_{a,b}$.
3. Finally we argue, that because $S_{a,b}$ is an optimum solution for the 4-cycle-breaking problem within $G_{a,b}$ and $F_{a,b}$ is a feasible solution for the same problem, we have:

$$w(S_{a,b}) \leq w(F_{a,b})$$

.

The statement in Lemma 4.5 holds for arbitrary values a, b. Therefore, in G, it holds that:

$$\forall i, j : w(S_{i,j}) \leq w(F_{i,j})$$

□

Result 4.6: Optimum solution bound for the whole graph (Weighted Case)

Let S_i be the union of optimal solutions defined on line 6 of Algorithm 4 for some shift i. Let $F \equiv OPT$ be the optimum solution in G and let $F_{i,j} = G_{i,j} \cap F$. For those sets it holds that:

$$w(S_i) \leq \sum_j w(S_{i,j}) \leq \sum_j w(F_{i,j}).$$

Proof. This result immediately follows from the fact that when we define S_i as the union of the subgraphs some vertices, specifically those on the overlapping layers, will have their weights added twice to the sum $\sum_j w(S_{i,j})$. If we inspect the LHS, however, each vertex and its weight will be counted only once towards $w(S_i)$. Therefore, it holds that $w(S_i) \leq \sum_j w(S_{i,j})$ and from Lemma 4.5 we chain the inequalities to get Result 4.6.

□

Lemma 4.7: Bound for the weight of the vertices on the boundary (Weighted Case)

Let $G = (V, E)$ be a planar graph and suppose we run a BFS from an arbitrary vertex. Let $w : V \to \mathbb{R}$ be a weight function. Let $F_i = F \cap \{\text{vertices at levels } i \bmod \ell\}$. The sets F_i are disjoint and $\bigcup_i F_i = F$.
We claim that:

$$\exists q \in \{0, 1, \ldots, \ell - 1\} : w(\sigma_2{}^{\bmod \ell}(q)) \leq \frac{2}{\ell} \cdot w(F)$$

Proof. Assume, by way of contradiction, that:

$$\forall i \in \{1, 2, \ldots, \ell - 1\}, w(\sigma_2{}^{\bmod \ell}(i)) > \frac{2}{\ell} \cdot w(F). \tag{2}$$

1. From the two properties in the definition of F_i, namely that

   $F_i \cap F_j = \emptyset, \forall i, j \in \{0, 1, 2, \ldots, \ell - 1\}, i \neq j$ and

   $F = \bigcup_i F_i \ \forall i \in \{0, 1, 2, \ldots \ell - 1\}$,

it follows that $\sum_{i=0}^{\ell-1} w({\sigma_2}^{\bmod \ell}(i)) = 2 \cdot w(F)$.

2. However, if inequality 2 holds, then $\sum_{i=0}^{\ell-1} w({\sigma_2}^{\bmod \ell}(i)) > \sum_{i=0}^{\ell-1} \frac{2}{\ell} \cdot w(F) = 2 \cdot w(F)$. Because each F_i is counted twice in the sum of step 2, then there is a contradiction between the sums in steps 1 and 2.
3. Because of the *pigeonhole principle*, there $\exists q$, such that $w({\sigma_2}^{\bmod \ell}(q)) \leq \frac{2}{\ell} \cdot w(F)$.

□

Lemma 4.8: Bound for the weight of the intersection sets (Weighted Case)

Let $F \equiv OPT$ be the optimal solution of the FVS-4CL problem. Let $F_{i,j} = F \cap G_{i,j}$ be the intersection sets with the subgraphs. Then we have:

$$\sum_j w(F_{i^*,j}) \leq w(F) + \frac{2}{\ell} \cdot w(F)$$

for some specific integer i^*.

Proof. The interior vertices of each subgraph have their weights counted only once towards the sum $\sum_j w(F_{i,j})$. The same does not hold for the vertices on the boundary, they are counted once in $F_{i,j}$ and once in $F_{i,j+1}$. To account for that we rewrite the sum on the LHS as:

1. $\sum_j w(F_{i,j}) = w(F) + w(\{\text{vertices on the boundary}\})$.
2. From Lemma 4.7 we have an integer $i^* \in \{1, 2, \ldots, \ell - 1\}$, such that:
   $w(OPT_{i^*}) + w(OPT_{i^*+1}) \leq \frac{2}{\ell} \cdot w(F)$.
3. From the previous two steps we get:

$$\sum_j w(F_{i,j}) = w(F) + w(\{\text{vertices on the boundary}\}) \leq \frac{2}{\ell} \cdot w(F)$$

□

Finally, we chain the inequalities we obtained in Lemma 4.5, Result 4.6, and Lemma 4.8 to argue that for some $i^* \in \{1, 2, \ldots, \ell - 1\}$:

$$w(S_{i^*}) \leq \sum_j w(S_{i^*,j}) \leq \sum_j w(F_{i^*,j}) \leq w(F) + \frac{2}{\ell} \cdot w(F)$$

.

Running time:

The only difference from the unweighted case comes from line 5 of Algorithm 4. We call a different subroutine, namely Algorithm 5. However, using Theorem 4.12, we get that the running time of the algorithm is still $O(f(3\ell + 5) \cdot \ell \cdot n) = O(f(3\ell + 5) \cdot n)$ for some computable function f.

THINGS TO PROVE/CLARIFY:

1. justification for outerplanarity (easy?) for each subgraph, i.e. $G_{i,j}$ is ℓ-outerplanar (because of BFS).
2. feasibility of the union solution, prove that S_i is indeed a solution, here make use of the number of overlap layers (medium)

3. clarify how we use the shift and slice variables, what does $\forall(i,j)$ mean, explain we actually talk about the subgraphs, what does i^* mean.
4. for lemma 4.7 and lemma 4.8,

4.3 Baker's technique for weighted FVS-BCL of arbitrary cycle length

In this subsection we generalize what we saw in subsection 4.2 and inspect what happens when we try to break all cycles of length ρ. To reiterate, our algorithm is only interested in breaking cycles of length ρ.

Algorithm 5 Baker's technique for the weighted FVS-BCL

Require: Planar graph $G = (V, E)$, parameter $\ell \leftarrow \frac{1}{\epsilon}$, $w : V \to \mathbb{R}$, ρ - integer specifying cycles of what length to break

Ensure: $(1+\epsilon)$-approximation for FVS-BCL

1: Perform BFS from some arbitrary vertex r
2: $S \leftarrow \emptyset$
   i: *shift*; j: *slice*
3: **for** each $i = 0$ to $\ell - 1$ **do**
4: Let $G_{i,j}$ be the subgraph induced on vertices at levels $j \cdot \ell + i$ through $(j+1) \cdot \ell + i + \lfloor \frac{\rho}{2} \rfloor$ for $0 \leq i < \ell$.
5: $\forall j$ compute $S_{i,j}$, the min. weighted FVS-BCL of $G_{i,j}$ using Algorithm 6 as a subroutine.
6: $\forall j$ let $S_i = \bigcup_j S_{i,j}$
7: $S \leftarrow S \cup S_i$
8: **end for**
9: **return** S_{i*} from S, such that $w(S_{i^*}) \leq w(S_i), \forall i \in \{1, 2, \ldots, \ell - 1\}$.

The changes between Algorithm 4 and Algorithm 5 pertain to lines 4-7. The main difference is on line 4 where we define the subgraphs we want to inspect. This change enables us to argue that the union of the local solutions, defined on line 6, is indeed a feasible solution for the whole graph.

Lemma 4.9: Optimum solution bound for FVS-ρCL

Let $G = (V, E)$ be a planar graph, $w : V \to \mathbb{R}$ be a weight function and ρ an integer. Let $F \equiv OPT$ be the optimum solution in the whole graph for FVS-ρ-BCL and $F_{i,j} = G_{i,j} \cap F$. Let $S_{i,j}$ be an optimum solution for $G_{i,j}$ for some shift i and slice j. We claim that:

$$w(S_i) \leq \sum_j w(S_{i,j}) \leq \sum_j w(F_{i,j}).$$

Proof. We first prove the second inequality by choosing arbitrary shift and slice variables (a, b) and observing that the inequality holds. The induced subgraph $G_{a,b}$ is defined on levels $b \cdot \ell + a$ through $(b+1) \cdot \ell + a + \lfloor \frac{\rho}{2} \rfloor$. In the next subsection, we argue that using Alg. 6, we get $S_{a,b}$, an optimal solution in $G_{a,b}$, while $F_{a,b}$ is feasible, but not necessarily optimal. Thus the following inequality holds:

$$\sum_j w(S_{i,j}) \leq \sum_j w(F_{i,j})$$

The first inequality holds, because the vertices on the boundaries are counted twice towards the RHS of the inequality and only once in the LHS. Therefore we have:

$$w(S_i) \leq \sum_j w(S_{i,j})$$

Chaining the inequalities we get

$$w(S_i) \leq \sum_j w(S_{i,j}) \leq \sum_j w(F_{i,j})$$

□

Lemma 4.10: Bound for the weight of the vertices on the boundary ρ-cycle length

Let $G = (V, E)$ be a planar graph and suppose we run a BFS from an arbitrary vertex.
Let $w : V \to \mathbb{R}$ be a weight function. Let $F_i = F \cap \{\text{vertices at levels } i \bmod \ell\}$. The sets F_i are disjoint and $\bigcup_i F_i = F$. Let $\sigma^{\ell}_{\lfloor \rho/2 \rfloor}(q) = \bigcup_q F_q \cup F_{q+1}, \ldots F_{q+\lfloor \rho/2 \rfloor} - 1$.
We claim that:

$$\exists q \in \{0, 1, \ldots, \ell - 1\} : w(\sigma_{\lfloor \rho/2 \rfloor}(q)) \leq \frac{\lfloor \rho/2 \rfloor}{\ell} \cdot w(F)$$

Proof. Assume, by way of contradiction, that:

$$\forall i \in \{0, 1, \ldots, \ell - 1\} : w(\sigma_{\lfloor \rho/2 \rfloor}(i)) > \frac{\lfloor \rho/2 \rfloor}{\ell} \cdot w(F) \tag{3}$$

1. In Lemma 4.10 we laid out two properties for F_i, namely that:

   $F_i \cap F_j = \emptyset, \forall i, j \in \{0, 1, 2, \ldots, \ell - 1\}, i \neq j$ and

   $F = \bigcup_i F_i \ \forall i \in \{0, 1, 2, \ldots \ell - 1\}$,

   it follows that $\sum_{i=0}^{\ell-1} w(\sigma_{\lfloor \rho/2 \rfloor}(i)) = \lfloor \rho/2 \rfloor \cdot w(F)$ as every set F_i is counted $\lfloor \rho/2 \rfloor$ times.

2. However, if inequality 3 holds, then:

   $$\sum_{i=0}^{\ell-1} w(\sigma_{\lfloor \rho/2 \rfloor}(i)) > \sum_{i=0}^{\ell-1} \frac{\lfloor \rho/2 \rfloor}{\ell} \cdot w(F) = \lfloor \rho/2 \rfloor \cdot w(F)$$

   There is a contradiction between the results in step one and two.

3. Therefore, we claim that:

   $$\exists q \in \{0, 1, \ldots, \ell - 1\} : w(\sigma_{\lfloor \rho/2 \rfloor}(q)) \leq \frac{\lfloor \rho/2 \rfloor}{\ell} \cdot w(F)$$

□

Lemma 4.11: Bound for the weight of the intersection sets

Let $F \equiv OPT$ be the optimal solution of the FVS-ρCL length problem for an integer $\rho \in \mathbb{N}_{\geq 3}$. Let $F_{i,j} = F \cap G_{i,j}$ be the intersection sets with the subgraphs. Then we have:

$$\sum_j w(F_{i^*,j}) \leq w(F) + \frac{\lfloor \rho/2 \rfloor}{\ell} \cdot w(F)$$

for some specific integer i^*.

Proof. 1. $\sum_j w(F_{i,j}) = w(F) + w(\{\text{vertices on the boundary}\})$.

2. From Lemma 4.10 we have that:

$$\exists i^* \in \{0, 1, \ldots, \ell - 1\} : w(\sigma_{\lfloor \rho/2 \rfloor}(i^*)) \leq \frac{\lfloor \rho/2 \rfloor}{\ell} \cdot w(F)$$

The vertices on the boundary are comprised of exactly $\lfloor \rho/2 \rfloor$ consecutive layers of vertices.

3. From the previous two steps we get:

$$\sum_j w(F_{i^*,j}) = w(F) + w(\{\text{vertices on the boundary}\}) \leq \frac{\lfloor \rho/2 \rfloor}{\ell} \cdot w(F)$$

□

Running time:

In Algorithm 5 which solves FVS-BCL the computation on line 5 dominates the running time. In planar graphs steps 1 and 9 take linear time to the input. Using Courcelle's theorem [30] we argue for the running time of line 5, which uses Alg. 6 as a subroutine. As we will say in the next section, the subroutine runs in time $f(||\phi||, t) \cdot n$ for some computable function f where a tree decomposition of width t is provided. Using a result from Bodlaender [29, Theorem 83] one can incur that the treewidth of each subgraph is $3\ell + 5$. Therefore the running time of Alg. 5 is $O(n) + O(f(3\ell + 5) \cdot \ell \cdot n) + O(n) = O(f(3\ell + 5) \cdot n)$ for some computable function f.

4.4 Exact algorithm for FVS-4CL using MSOL

In this subsection we are looking for an algorithm that finds exact solutions to the FVS-4CL problem in each subgraph. For each subgraph we can find the treewidth.

We take a logical approach that was introduced by Courcelle [30]. He shows that graph properties, that can be stated in monadic second-order logic (MSOL), can be solved in linear time for graphs of bounded treewidth. In the following two subsections we briefly explain MSOL and then apply it to our problem. We follow a survey on the matter in Satzinger [31].

4.4.1 MSOL - a language for graph problems

We want to introduce a language that translates graph problems to formulas, therefore reducing the problem to a satisfiability problem. Consequently, our goal is to find a formula Φ with the property:

The problem Π is solvable for a graph G *iff* $G \models \Phi$, i. e. the formula Φ is satisfied by the graph G.

Problem Π is solvable in polynomial (in this case linear) time for graphs of treewidth at most k.

Monadic second-order logic is an extension of first-order logic. Additionally to having object variables $x, y \ldots$, logical connectives $\neg, \wedge, \vee, \neg, \rightarrow, \longleftrightarrow$ and quantifiers for object variables $\forall x, \exists x$, MSOL considers sets of object variables $X, Y \ldots$, and relation to them via $\in$.

Whenever graph problems are involved, typically we express solution through vertices and edges. Additionally, as we mentioned, we can construct sets of vertices and edges. Therefore we have the following machinery:

- Object variables: *vertices*: $v_1, v_2, \ldots$ and *edges*: $e_1, e_2, \ldots$.
- Set variables: sets of vertices $V_1, V_2, \ldots$ and sets of edges $E_1, E_2, \ldots$.

- Binary relation $\in$: {object variable} $\times$ {set variable} $\rightarrow \{0,1\}$. Therefore $v \in V$ *iff* v is an element of the corresponding set V.
- The $Adj(e, v_i, v_j)$ relation. It detects whether edge e is an edge from vertex v_i to vertex v_j where $v_i \neq v_j$.

With this machinery, one obtains a first-order language suitable for expressing relations in a graph. To extend this to a traditional second-order logic language, one must add quantifications over set variables.

- Quantification over set variables: $\forall V_i, \forall E_i$ and $\exists V_i, \exists E_i$.

4.4.2 Exact algorithm for unweighted FVS-4CL using MSOL

Courcelle's work was extended to extended monadic second-order logic [32, 33, 34]. These extensions allow to optimize over the size and weight of a free set variable.

$$\begin{aligned}
&\min_{F \subseteq V} |F| : \\
&\quad \wedge \forall vuwz : \\
&\quad v \in (V \backslash F) \wedge u \in (V \backslash F) \wedge w \in (V \backslash F) \wedge z \in (V \backslash F) \\
&\quad \wedge v \neq w \wedge u \neq z \\
&\quad \wedge \neg((v,u) \in E \wedge (u,w) \in E \wedge (w,z) \in E \wedge (v,z) \in E)
\end{aligned} \tag{4}$$

4.4.3 Exact algorithm for weighted FVS-4CL using MSOL

The minimum weight FVS-4CL can be expressed as:

$$\begin{aligned}
&\min_{F \subseteq V} w(F) : |F| \leq k \\
&\quad \wedge \forall vuwz : \\
&\quad v \in (V \backslash F) \wedge u \in (V \backslash F) \wedge w \in (V \backslash F) \wedge z \in (V \backslash F) \\
&\quad \wedge v \neq w \wedge u \neq z \\
&\quad \wedge \neg((v,u) \in E \wedge (u,w) \in E \wedge (w,z) \in E \wedge (v,z) \in E)
\end{aligned} \tag{5}$$

4.4.4 Exact algorithm for weighted FVS-BCL using MSOL

The minimum weight FVS-ρCL can be expressed as:

$$\begin{aligned}
&\min_{F \subseteq V} w(F) : |F| \leq k \\
&\quad \wedge \forall v_1 v_2 \ldots v_\rho : \\
&\quad v_1 \in (V \backslash F) \wedge v_2 \in (V \backslash F) \cdots \wedge v_\rho \in (V \backslash F) \\
&\quad \wedge (v_1 \neq v_2 \wedge v_1 \neq v_3 \wedge \cdots \wedge v_1 \neq v_\rho) \\
&\quad \wedge (v_2 \neq v_1 \wedge v_2 \neq v_3 \wedge \cdots \wedge v_2 \neq v_\rho) \\
&\quad \ldots \\
&\quad \wedge (v_\rho \neq v_1 \wedge v_\rho \neq v_2 \wedge \cdots \wedge v_\rho \neq v_{\rho-1}) \\
&\quad \wedge \neg((v_1,v_2) \in E \wedge (v_2,v_3) \in E \wedge \cdots \wedge (v_{\rho-1},v_\rho) \in E \wedge (v_\rho,v_1) \in E)
\end{aligned} \tag{6}$$

Theorem 4.12: Courcelle's theorem [30]

Assume that ϕ is a MSOL formula and G is an n-vertex graph, and there is an evaluation of all of the free variables of ϕ. Suppose, that a tree decomposition of G of width t is provided. Then there exists an algorithm that verifies that ϕ is satisfied in G in time $f(||\phi||, t) \cdot n$, for some computable function f.

4.5 Exact algorithm for FVS-4CL using dynamic programming

In order to find an algorithm that solves FVS-4CL, one has to define the following notions. For a more thorough explanation of those concepts, see Bodlaender [35, Section 4.].

1. Define the notion of a *solution*. In our case for the FVS-4CL problem, the solution for a graph G is a set F, such that $G - F$ does not contain any cycles of length four.
2. Define the notion of a *partial solution*. For the subgraph $G_i = (V_i, E_i)$, partial solutions F_i are restrictions of solutions given by the subsets $F_i \subseteq V_i$.
3. Define the notion of *extension of a partial solution*. A solution F is an extension of a partial solution F_i for G_i if $F \cap V_i = F_i$.
4. Define the notion of a *characteristic* of a partial solution. For the set X_i the vertices are partitioned twofold.
   (a) The class I corresponds to all vertices that are in the partial solution.
   (b) The class $Flag$ is comprised of all of the pairs of vertices $v, u \in X_i : (v, u) \in Flag$ if they have a common neighbour $w \in V_i \backslash X_i$, such that $w \notin I$.

   Let $v, u \in X_i$.

$$ch(G_i, F_i) = (v \in I, (v, u) \in Flag)$$

5. Define the notion of a *full set of characteristics*.

5 m-stitching and FVS-BCL-m-stitching

In this section we initiate the study of certified algorithms for FVS-BCL. We introduce some key concepts that allow us to construct certified algorithms in general. Some of them include *m-stitching* and the notion of Π-*m-stitching*. We refer to the literature in [9, 36].

Unless specified otherwise, let Π be a vertex-minimization problem.

5.1 Algorithms for m-stitching and FVS-m-stitching

As we mentioned before, in this section we refer to Bumpus et al. [9] and a Master's thesis on the subject by Venne [36] with more examples.

The restriction we imposed on the Feedback Vertex Set problem enables us to look for optimal solutions in a local space. This is what m-stitching and Π-m-stitching hint at. Here, the the variable m is a positive integer that depends on the vertex optimization problem Π at hand. In our case this is tied to the ρ argument, i. e. the length of the cycles we are trying to break.

To introduce Π-m-stitching, one needs to first understand m-stitching and m-stitchable problems. In the stitching operation we are given solutions S_1 and S_2 for a vertex optimization problem, along with an induced subgraph J of G. Intuitively, we take the portion of S_1, that is not a part of J and "stitch" onto it some part of S_2 along the subgraph J.

The formal definition is as follows:

Def. 5.1: m-stitching

Assume $m \geq 0$ is an integer, J is an induced subgraph of G, and $S_1, S_2 \subseteq V(G)$. Then we define the *m-stitch* of S_2 onto S_1 along J as the set:

$$S_3 := (S_1 \backslash J) \cup (S_2 \cap N_G^m[J]).$$

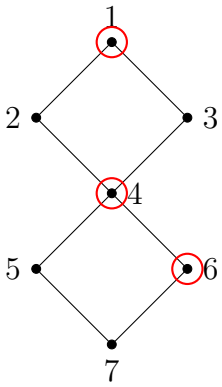

Figure 3: Vertex set S_1

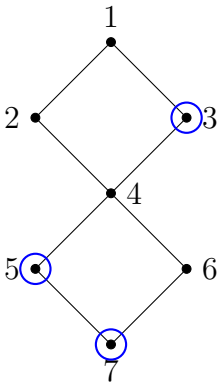

Figure 4: Vertex set S_2

In the following we show the subgraph J we have chosen and the 2-neighbourhood of the subgraph.

Also, in Fig. 6 we show the first step of the m-stitch operation, namely removing the vertices of J from the set S_1.

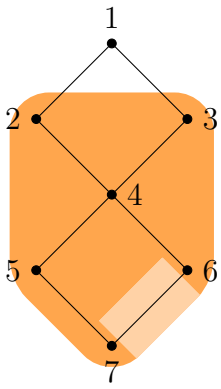

Figure 5: Subgraph J of G in light orange and $N_G^2[J]$ in darker orange

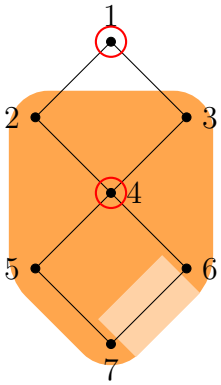

Figure 6: Remove J from S_1

What follows is the second step in obtaining set S_3 in Fig. 7 and the stitched set in Fig. 8.

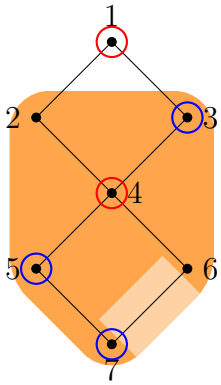

Figure 7: Add $S_2 \cap N_G^2[J]$

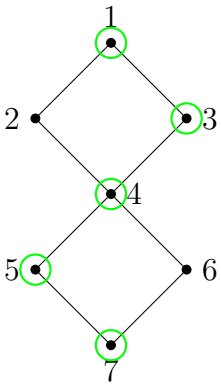

Figure 8: Set S_3

A problem Π is *m-stitchable* if the set of feasible solutions are closed under the m-stitching operator. We hope that when we take the union in 5.1 and arrive at the set S_3, this would too be a feasible solution.

Def. 5.2: m-stitchable

For an integer $m \geq 0$, induced subgraph J and feasible solutions $S_1, S_2 \subseteq V(G)$ of a vertex optimization problem Π we have that the m-stitching of S_2 onto S_1 along J, S_3 is a feasible solution to Π on G.

$$S_3 := (S_1 \backslash J) \cup (S_2 \cap N_G^m[J]).$$

Now that we have a definition for problems Π that are m-stitchable, we introduce Π-m-stitching as defined in [9] and afterwards apply it to the current problem.

Algorithm. 5.3: Π-m-stitching

Input: an instance $(G, w : V(G) \to \mathbb{N})$ to an m-stitchable vertex optimization problem Π along with a solution S_1 and a vertex set $J \subseteq V(G)$.
Task: find a feasible solution S' to Π on G, such that for all other feasible solutions S^* it holds that $w(S') \leq w\left(S^* \oplus_{G,J}^2 S_1\right)$.

Before we proceed with the main theorem that we use to construct a certified algorithm we need one more definition.

Def. 5.4: Guessable

A problem Π is guessable if there is an algorithm that outputs a feasible solution with no requirement for optimality in polynomial time.

The following theorem is the main result in Bumpus et al. [9, Theorem 1.7].

Theorem. 5.5:

Let $\mathbb{G}$ be a minor-closed graph class whose local tree-width is bounded above by a linear function of the form $g : r \to \lambda \cdot r$ (fixed $\lambda \in \mathbb{R}$ and $r \in \mathbb{N}$). If Π is a vertex-minimization problem and it holds that:

1. Π is guessable.
2. Π is m-stitchable.
3. There exists an algorithm A_Π which solves Π-m-stitching in time $f(t) \cdot |V(G)|^{\mathbb{O}(1)}$, where t = $\mathbf{tw}(G\,[N_G^m(J)])$ and f is a computable function;

then, for each $\epsilon > 0$ there is a $(1+\epsilon)$-certified algorithm for Π that runs in time $f(\lambda \cdot m/\epsilon) \cdot |V(G)|^{\mathbb{O}(1)}$.

The proof of theorem 5.5 is beyond the scope of this paper. In this section we only apply the three steps, mentioned to the FVS-BCL problem without generalizing to other vertex-minimization problems.

Firstly, we take a look at the guessable property.

Lemma 5.6: FVS-BCL is guessable

For any planar graph G, the set $V(G)$ is a feasible solution to FVS-BCL.

Proof. The graph $G - X$ where X is the set of all vertices $V(G)$ is the empty graph, therefore there are no 4-cycles left, after selecting that set. □

Lemma 5.7: FVS-BCL is 2-stitchable

Let (G, w) be any instance to the FVS-4CL problem, $J \subseteq V(G)$, some subset of the vertices in the graph and S_1 and S_2 any two feasible solutions to the problem in G. Then the set S_3:

$$S_3 := (S_1 \backslash J) \cup (S_2 \cap N_G^2\,[J]).$$

is a feasible solution to the problem.

Proof. Let the vertices $\{v_1, v_2, v_3, v_4\}$ form a 4-cycle. We inspect the possible cases for the 4-cycle $v_1v_2v_3v_4$. In order to prove 5.7 we have to prove that there exists a $v \in \{v_1, v_2, v_3, v_4\}$, such that $v \in S_3$.

- $\exists v' \in \{v_1, v_2, v_3, v_4\}$, such that $v' \in J$. We argue in three steps.
  (a) Because S_2 is a feasible solution, then $\exists v'' \in \{v_1, v_2, v_3, v_4\}$, such that $v'' \in S_2$.
  (b) Because there $\exists v' \in \{v_1, v_2, v_3, v_4\} \in J$, then it also holds that $\forall v' \in \{v_1, v_2, v_3, v_4\}, v' \in N_G^2[J]$. From that and the previous point it follows that $v'' \in S_2 \cap N_G^2[J]$.
  (c) Finally, $S_2 \cap N_G^2[J] \subseteq S_3$, therefore $v'' \in S_3$.
- $\forall v' \in \{v_1, v_2, v_3, v_4\}$, it holds that $v' \notin J$. This case follows from the feasibility of S_1.

(a) Because S_1 is a feasible solution, then $\exists v'' \in \{v_1, v_2, v_3, v_4\}$, such that $v'' \in S_1$.

(b) From $\forall v' \in \{v_1, v_2, v_3, v_4\} \notin J$ and the previous point, it follows that $v'' \in S_1 \backslash J$.

(c) Because $S_1 \backslash J \subseteq S_3$, then $v'' \in S_3$.

□

Now we describe an algorithm to find a feasible solution of minimum weight.

Algorithm 6 Stitch-FVS-4CL

Require: Vertex-weighted planar graph $(G, w : V(G) \to \mathbb{N})$, a feasible solution S_1 on G and a vertex set $J \subseteq V(G)$.

Ensure: a feasible solution S' on G, such that for all other feasible solutions S^*, we have $w(S') \leq w(S^* \oplus^2_{G,J} S_1)$.

1: Let $H \leftarrow G[N^2_G[J] \backslash (S_1 \backslash J)]$.
2: Let S_2 be the output of algorithm A on input (H, w).
3: **if** $w(S_2 \oplus^2_{G,J} S_1) < w(S_1)$ **then**
4: **return** $S_2 \oplus^2_{G,J} S_1$
5: **else**
6: **return** S_1
7: **end if**

The algorithm A on line 6 refers to calling Algorithm 6 that takes as input a graph that has bounded treewidth.

Lemma 5.8: FVS-4CL-2-stitching

Let A be an algorithm that solves FVS-4CL on any vertex-weighted instance $(G, w : V(G) \to \mathbb{N})$ in time $f(tw(G)) \cdot |V(G)|^{O(1)}$. Then Algorithm 6 solves FVS-4CL-2-stitching in time $f(tw(N^2_G[J])) \cdot V(G)^{O(1)}$.

Note that it is not obvious that the set $S_2 \oplus^2_{G,J} S_1$ is a feasible solution in G as S_2 is not necessarily a feasible solution in G. If the last was the case, then the feasibility of the $S_2 \oplus^2_{G,J} S_1$ would follow from Lemma 5.7. Therefore we first prove feasibility of $S_2 \oplus^2_{G,J} S_1$. Afterwards, we prove that it is of minimum weight.

Proof. We prove the feasibility of $S_2 \oplus^2_{G,J} S_1$ by case analysis on any four vertices $v_1v_2v_3v_4$ that form a 4-cycle. Remember that $S_2 \oplus^2_{G,J} S_1 = (S_1 \backslash J) \cup (S_2 \cap N^2_G[J])$.

1. $\forall v' \in \{v_1, v_2, v_3, v_4\} : v' \in V(G) \backslash J$.

   Firstly, because of the feasibility of S_1, there $\exists v'' \in \{v_1, v_2, v_3, v_4\} : v'' \in S_1$.

   Secondly, $\forall v' \in \{v_1, v_2, v_3, v_4\}$, it holds that $v' \in V(G) \backslash J$.

   Finally from the first two conditions, using the property $A \backslash B = A \cap B^c$, it holds that $v'' \in S_1 \backslash J$. From this and $S_1 \backslash J \subseteq S_2 \oplus^2_{G,J} S_1$, it follows that $S_2 \oplus^2_{G,J} S_1$ is feasible in G.

   In Fig. 9 we show (a part of) an example graph G and the feasible solution S_1 for it. In Fig. 10, one can see that 4-cycles that do not have vertices in J, are a part of $S_1 \backslash J \subseteq S_2 \oplus^2_{G,J} S_1$.

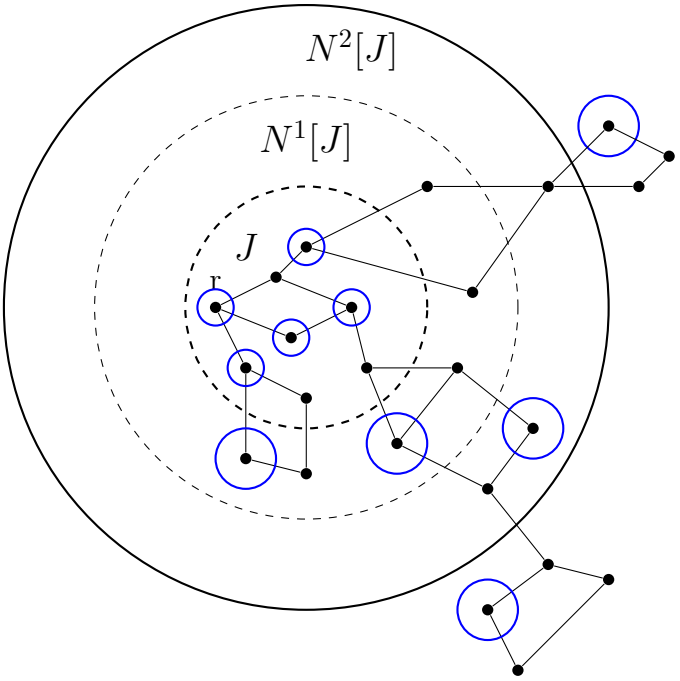

Figure 9: Sets $J \subseteq N_G^1[J] \subseteq N_G^2[J] \subseteq G$; Feasible solution S_1 colored in blue.

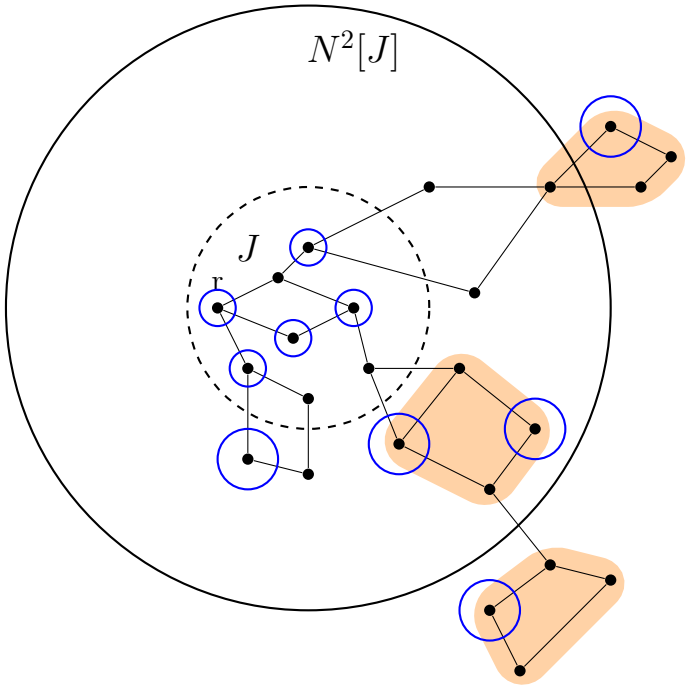

Figure 10: $\forall v \in \{v_1, v_2, v_3, v_4\} : v \notin J$

2. $\exists v' \in \{v_1, v_2, v_3, v_4\} : v' \in J$. This also means that in this case $\forall v' \in \{v_1, v_2, v_3, v_4\} : v' \in N_G^2[J]$. There are two cases to consider again. We know there $\exists v'' \in \{v_1, v_2, v_3, v_4\}$, such that $v'' \in S_1$, we still have not considered whether or not it is a part of J.

   (a) $\exists v \in S_1 : v \notin J$:

   In this case we can directly apply the previous point. From the properties of the set difference operator we get: $v \in S_1 \backslash J \subseteq S_2 \oplus_{G,J}^2 S_1$. Consult Fig. 11 to see that there exists a vertex $v_{1,4}$ that is a part of the 4-cycle, $v_{1,4} \notin J$, therefore $v_{1,4} \in S_1 \backslash J \subseteq S_2 \oplus_{G,J}^2 S_1$. This means that in the new set $v_{1,4}$ will break that cycle.

   (b) $\forall v \in S_1 : v \in J$:

   In line 1 of Algorithm 6, we have the following construction of $H \leftarrow G[N_G^2[J] \backslash (S_1 \backslash J)]$.

$$V(H) = N_G^2[J] \backslash (S_1 \backslash J) \tag{7}$$
$$= N_G^2[J] \cap J \cup (N_G^2[J] \backslash S_1) \tag{8}$$
$$= J \cup (N_G^2[J] \backslash S_1) \tag{9}$$

   However, $\forall v \in \{v_1, v_2, v_3, v_4\} : v \in S_1 \wedge v \in J$. Therefore $\{v_1, v_2, v_3, v_4\} \subseteq V(H)$. This means that in line 2 of Algorithm 6 we will add some $v^* \in \{v_1, v_2, v_3, v_4\}$ to S_2.

   Therefore $v \in S_2$, which has the same meaning as $v \in S_2 \cap N_G^2[J]$, and because $S_2 \cap N_G^2[J] \subseteq S_2 \oplus_{G,J}^2 S_1$, we get $v \in S_2 \oplus_{G,J}^2 S_1$.

   Consult Fig. 12.

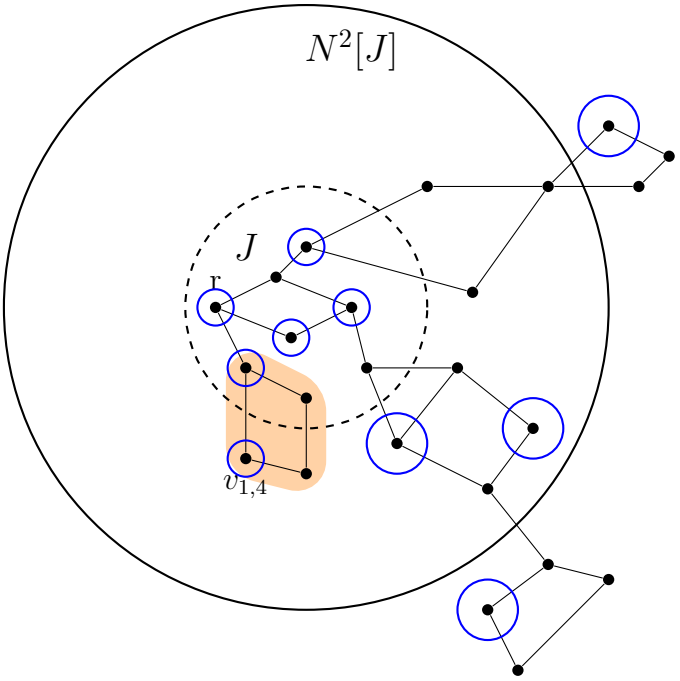

Figure 11: $\exists v \in S_1 : v \notin J$ (Remember we are in the case where $\exists v \in J$)

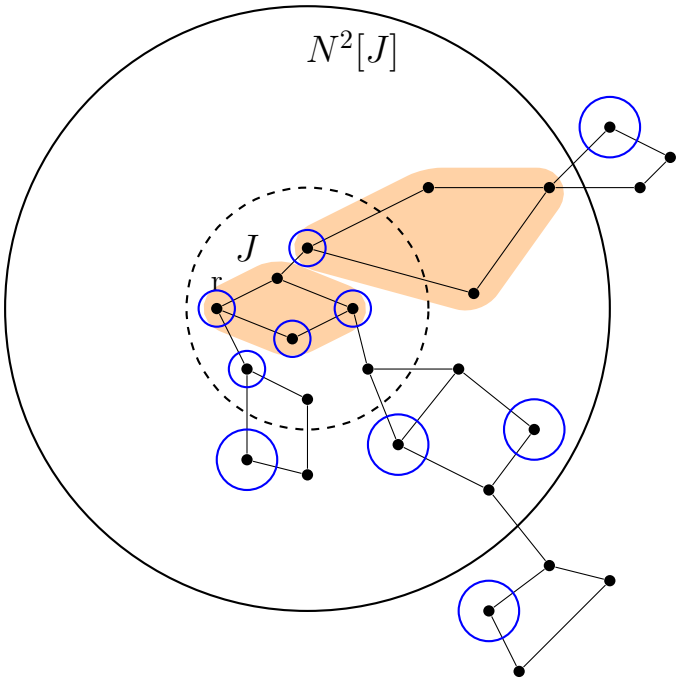

Figure 12: $\forall v \in S_1 : v \in J$

The proof of optimality of the set $S_2 \oplus^2_{G,J} S_1$ is similar to the one in Bumpus et al.[9, Lemma 3.2]. It is important to note that we can use the same proof, because of a property of the FVS-4CL problem, namely that a 4-cycle breaking set is closed under taking induced subgraphs.

Remember, we are still trying to prove what we mentioned in Algorithm 5.3, namely that some feasible solution S' (which in our case is the $S_2 \oplus^2_{G,J} S_1$), is of minimum weight, that is, for all other feasible solutions S^*, it holds that $w(S') \leq w(S^* \oplus^2_{G,J} S_1)$.

Assume by way of contradiction that there is a feasible solution S_3 for G, such that $w(S_3 \oplus^2_{G,J} S_1) < w(S_2 \oplus^2_{G,J} S_1)$. Then it would follow that:

$$
\begin{aligned}
w|_{V(F)}(S_2) &= w(S_2) && \text{since } S_2 \subseteq V(F) \\
&= w(S_2 \cap N^2_G[J]) && \text{since } V(F) \subseteq N^2_G[J] \\
&= w(S_1 \setminus J) + w(S_2 \cap N^2_G[J]) - w(S_1 \setminus J) \\
&= w((S_1 \setminus J) \cup (S_2 \cap N^2_G[J])) - w(S_1 \setminus J) && \text{since } V(F) \cap (S_1 \setminus J) = \emptyset \\
&= w(S_2 \oplus^2_{G,J} S_1) - w(S_1 \setminus J) && \text{by definition of stitch} \\
&> w(S_3 \oplus^2_{G,J} S_1) - w(S_1 \setminus J) && \text{by assumption on } S_3 \\
&= w((S_1 \setminus J) \cup (S_3 \cap N^2_G[J])) - w(S_1 \setminus J) && \text{by definition of stitch} \\
&= w((S_3 \cap N^2_G[J]) \setminus (S_1 \setminus J)) && \text{since } w(A \cup B) - w(A) = w(B \setminus A) \\
&= w(S_3 \cap (N^2_G[J] \setminus (S_1 \setminus J))) && \text{since } (A \cap B) \setminus C = A \cap (B \setminus C) \\
&= w(S_3 \cap V(F)) && \text{by definition of } F \\
&= w|_{V(F)}(S_3 \cap V(F))
\end{aligned}
$$

□

Running time:

The running time of the algorithm is dominated by the call of algorithm A. Therefore, for an integer m

and some computable function f, the running time is $O(tw(N_G^m[J])) \cdot |V(G)|^{O(1)}$.

6 Certified Algorithm

In this section we present an algorithm that finds γ-certified solutions to the FVS-4CL problem. It is similar to the one in Bumpus et al. [9, 4.2.1], but is adopted to our problem. The proof that Algorithm 7 is a $(1+\epsilon)$-certified algorithm is beyond the scope of this paper. This is because the proof is similar to the one in Bumpus et al. [9, Section 4.2.3].

However, we still present the algorithm, as this brings more clarity on how exactly one obtains $(1+\epsilon)$-certified solutions, and, on a separate note, we want to argue about the running time of Algorithm 7.

6.1 Certified Algorithm for FVS-4CL

Before we give the algorithm we will define some concepts, that we will make use of.

Def. 6.1: κ-layered subgraph

Let G be a planar graph and κ a natural number. Let J be the set of vertices of any κ subsequent layers, where the layers are a result of a BFS run. Then the graph $G[V(J_\kappa)]$ is a κ-layered subgraph. We denote such a subgraph G_κ. If the set of vertices J is made of κ subsequent layers, then we denote it as J_κ.

Notice that Algorithm 6 takes as input a graph and a weighted function (G, w), a feasible solution S and some vertex set J.

Algorithm 7 Certified Algorithm

Require: Planar graph $G = (V, E)$, weight function $w : V(G) \to \mathbb{N}$, parameter ϵ
Ensure: Solution S to FVS-4CL in G, $(1+\epsilon)$-perturbed weights w' of w that certify S.

1: Perform BFS from some arbitrary vertex r .
2: $L_i = \{v \in V(G) \mid d(v, r) = i\}$ for $i = 0, \dots d_{\max}$.
3: Compute a feasible solution S to G according to Def. 5.4.
4: Let $k \leftarrow \lceil \frac{2m}{\epsilon} \rceil + 2m$.
5: **while** there exists a subgraph G_κ of width $k - 2m$ such that

$$w(A_{\text{FVS-4CL}}((G, w), S, J_\kappa)) < w(S)$$

do
6: $S \leftarrow A_{\text{FVS-4CL}}((G, w), S, J_\kappa)$
7: **end while**
8: $w' : V(G) \to \mathbb{N}$,

$$w' : v \to \begin{cases} w(v), & \text{if } v \in S \\ (1+\epsilon)w(v), & \text{otherwise.} \end{cases}$$

9: **return** (S, w').

The first stage consists of performing BFS from an arbitrary vertex r. This is done on lines 1,2 and is vital when later we have to choose a vertex set J_κ.

In the second stage, we compute a feasible solution S, set J_κ[1] and loop through the layers looking for

[1] A word on the argument k and the value that we set it to. In our case, for the FVS-4CL problem, m is equal to two. Therefore, $k = \lceil \frac{2m}{\epsilon} \rceil + 2m \geq 5$. Notice that the set J_κ is of width $k - 2m$, therefore the m-neighbourhood (2-neighbourhood) is of width $k - 2m + 2m = k$. Therefore, in our case, the graphs within we stitch are of width $k = \lceil \frac{2m}{\epsilon} \rceil + 2m \geq 5$. Take for

a better solution. The while statement is the one that dominates running time and provides us with an ultimately inclusion-minimal solution S on (G, w). Note that in the worst case there will be $W \cdot n$ calls to line 5, where $W = \sum_{v \in V(G)} w(v)$. This is because in each iteration we improve the value of the solution by at least 1, notice that we have used natural numbers for the weight functions.

In the final stage, we obtain a weight function for which we claim that Algorithm 7 is $(1 + \epsilon)$-certified.

Running time:

The running time of Algorithm 7 is clearly dominated by the running time incurred in the second stage. As we mentioned, there are at most $W \cdot n$ calls to Algorithm 6. It follows that the running time of the algorithm is $O()$.

example vertex $v_{2,1}$ in fig. 13. In order to find every cycle that it is a part of, one would need to consider the layer that it is in, two layers to the left and two layers to the right.

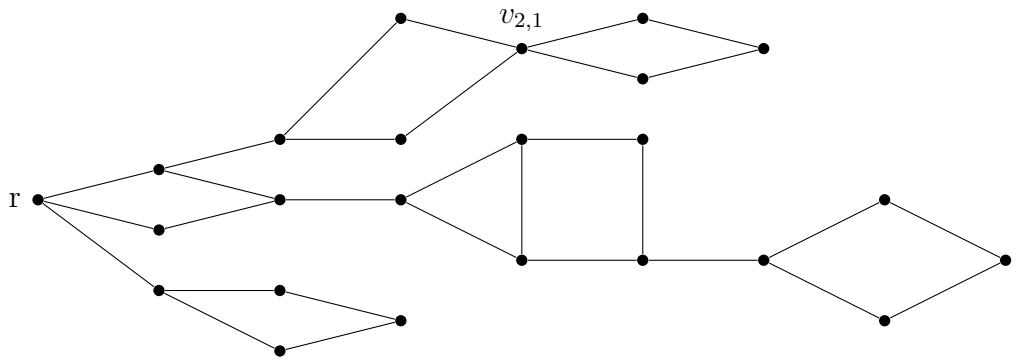

Figure 13: Layer decomposition of graph in Fig. 9

References

[1] R. M. Karp, *Reducibility among Combinatorial Problems*. Boston, MA: Springer US, 1972, pp. 85–103. [Online]. Available: https://doi.org/10.1007/978-1-4684-2001-2_9

[2] A. Becker and D. Geiger, "Optimization of pearl's method of conditioning and greedy-like approximation algorithms for the vertex feedback set problem," *Artificial Intelligence*, vol. 83, no. 1, pp. 167–188, 1996. [Online]. Available: https://www.sciencedirect.com/science/article/pii/0004370295000046

[3] V. Bafna, P. Berman, and T. Fujito, "A 2-approximation algorithm for the undirected feedback vertex set problem," *SIAM Journal on Discrete Mathematics*, vol. 12, no. 3, pp. 289–297, 1999. [Online]. Available: https://doi.org/10.1137/S0895480196305124

[4] F. A. Chudak, M. X. Goemans, D. S. Hochbaum, and D. P. Williamson, "A primal–dual interpretation of two 2-approximation algorithms for the feedback vertex set problem in undirected graphs," *Operations Research Letters*, vol. 22, no. 4, pp. 111–118, 1998. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0167637798000212

[5] B. S. Baker, "Approximation algorithms for NP-complete problems on planar graphs," *J. ACM*, vol. 41, no. 1, p. 153–180, jan 1994. [Online]. Available: https://doi.org/10.1145/174644.174650

[6] E. D. Demaine and M. T. Hajiaghayi, "Bidimensionality: new connections between fpt algorithms and ptass." in *SODA*, vol. 5, 2005, pp. 590–601.

[7] K. Makarychev and Y. Makarychev, *Perturbation Resilience*. Cambridge University Press, 2021, p. 95–119.

[8] H. Angelidakis, P. Awasthi, A. Blum, V. Chatziafratis, and C. Dan, "Bilu-linial stability, certified algorithms and the independent set problem," *arXiv preprint arXiv:1810.08414*, 2018.

[9] B. M. Bumpus, B. M. P. Jansen, and J. Venne, "Fixed-Parameter Tractable Certified Algorithms for Covering and Dominating in Planar Graphs and Beyond," in *19th Scandinavian Symposium and Workshops on Algorithm Theory (SWAT 2024)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), H. L. Bodlaender, Ed., vol. 294. Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2024, pp. 19:1–19:16. [Online]. Available: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.SWAT.2024.19

[10] Y. Bilu and N. Linial, "Are stable instances easy?" *Combinatorics, Probability and Computing*, vol. 21, no. 5, pp. 643–660, 2012.

[11] Y. Bilu, A. Daniely, N. Linial, and M. Saks, "On the practically interesting instances of maxcut," *arXiv preprint arXiv:1205.4893*, 2012.

[12] K. Makarychev, Y. Makarychev, and A. Vijayaraghavan, "Bilu–linial stable instances of max cut and minimum multiway cut," in *Proceedings of the twenty-fifth annual ACM-SIAM symposium on Discrete algorithms*. SIAM, 2014, pp. 890–906.

[13] P. Awasthi, A. Blum, and O. Sheffet, "Center-based clustering under perturbation stability," *Information Processing Letters*, vol. 112, no. 1-2, pp. 49–54, 2012.

[14] M. F. Balcan and Y. Liang, "Clustering under perturbation resilience," *SIAM Journal on Computing*, vol. 45, no. 1, pp. 102–155, 2016.

[15] M.-F. Balcan, N. Haghtalab, and C. White, "k-center clustering under perturbation resilience," *arXiv preprint arXiv:1505.03924*, 2015.

[16] M. Mihalák, M. Schöngens, R. Šrámek, and P. Widmayer, "On the complexity of the metric tsp under stability considerations," in *SOFSEM 2011: Theory and Practice of Computer Science: 37th Conference on Current Trends in Theory and Practice of Computer Science, Novỳ Smokovec, Slovakia, January 22-28, 2011. Proceedings 37*. Springer, 2011, pp. 382–393.

[17] K. Makarychev, "13 bilu-linial stability," *Perturbations, Optimization, and Statistics*, p. 375, 2017.

[18] J. A. Bondy, U. S. R. Murty *et al.*, *Graph theory with applications*. Macmillan London, 1976, vol. 290.

[19] B. Korte and J. Vygen, "Combinatorial optimization: Theory and algorithms," 2007. [Online]. Available: https://api.semanticscholar.org/CorpusID:265900003

[20] D. P. Williamson and D. B. Shmoys, *The design of approximation algorithms*. Cambridge university press, 2011.

[21] M. Cygan, F. V. Fomin, Ł. Kowalik, D. Lokshtanov, D. Marx, M. Pilipczuk, M. Pilipczuk, and S. Saurabh, *Parameterized algorithms*. Springer, 2015, vol. 5, no. 4.

[22] M. Cesati and L. Trevisan, "On the efficiency of polynomial time approximation schemes," *Information Processing Letters*, vol. 64, no. 4, pp. 165–171, 1997.

[23] M. Marzban and Q.-P. Gu, "Computational study on a ptas for planar dominating set problem," *Algorithms*, vol. 6, no. 1, pp. 43–59, 2013. [Online]. Available: https://www.mdpi.com/1999-4893/6/1/43

[24] R. Diestel, "Graph theory electronic edition 2000," 2000.

[25] R. Halin, "S-functions for graphs," *Journal of geometry*, vol. 8, pp. 171–186, 1976.

[26] N. Robertson and P. Seymour, "Graph minors. ii. algorithmic aspects of tree-width," *Journal of Algorithms*, vol. 7, no. 3, pp. 309–322, 1986. [Online]. Available: https://www.sciencedirect.com/science/article/pii/0196677486900234

[27] D. Eppstein, "Diameter and treewidth in minor-closed graph families," *Algorithmica*, vol. 27, pp. 275–291, 2000.

[28] E. D. Demaine and M. T. Hajiaghayi, "Equivalence of local treewidth and linear local treewidth and its algorithmic applications," 2003.

[29] H. L. Bodlaender, "A partial k-arboretum of graphs with bounded treewidth," *Theoretical computer science*, vol. 209, no. 1-2, pp. 1–45, 1998.

[30] B. Courcelle, "The monadic second-order logic of graphs. i. recognizable sets of finite graphs," *Information and computation*, vol. 85, no. 1, pp. 12–75, 1990.

[31] C. Satzinger, "On tree-decompositions and their algorithmic implications for bounded-treewidth graphs," Ph.D. dissertation.

[32] S. Arnborg, J. Lagergren, and D. Seese, "Easy problems for tree-decomposable graphs," *Journal of Algorithms*, vol. 12, no. 2, pp. 308–340, 1991.

[33] R. B. Borie, R. G. Parker, and C. A. Tovey, "Deterministic dcomposition of recursive graph classes," *SIAM Journal on Discrete Mathematics*, vol. 4, no. 4, pp. 481–501, 1991.

[34] B. Courcelle and M. Mosbah, "Monadic second-order evaluations on tree-decomposable graphs," *Theoretical Computer Science*, vol. 109, no. 1-2, pp. 49–82, 1993.

[35] H. L. Bodlaender, "Treewidth: Algorithmic techniques and results," in *International symposium on mathematical foundations of computer science*. Springer, 1997, pp. 19–36.

[36] J. Venne, "Designing (1)-certified algorithms for planar optimiza-tion problems with local feasibility properties," 2022.